

Rapport SAE2 Initiation au développement

Auteurs	Date
Boussaid Ilian G3SIB Lierley Kenzo G3S1B	Janvier 2025

Dans ce document, vous trouverez les spécifications des fonctions de tri, filtre et recherches qui ont permis de constituer les codes pour la SAE1. Seront abordés les thèmes des complexités théoriques, des types de fonctions, du calcul théorique des opérations élémentaires de ces fonctions, et enfin les codes permettant de vérifier les calculs théoriques de ceux-ci.

Sommaire

1. Algorithme de tri	2
1.1 Spécification (nom de la fonction, documentation)	2
1.2 Complexité théorique (ordre de grandeur et calcul en fonction de n).....	2
1.3 Code pour l'évaluation pratique.....	4
2. Algorithme de filtre	5
2.1 Spécification (nom de la fonction, documentation)	5
2.2 Complexité théorique (ordre de grandeur et calcul en fonction de n).....	5
2.3 Code pour l'évaluation pratique.....	7
3. Algorithmes de recherches.....	8
3.1 Spécification formelle	8
3.2 Complexité théorique des fonctions de recherche	8
3.3 Dénombrement des opérations élémentaires des fonctions de recherche	9
3.3.1 Dénombrement sur la fonction recherche_client.....	9
3.3.2 Dénombrement sur la fonction recherche_client_tel	11
3.3 Code des fonctions d'opérations formelles	11

1. Algorithme de tri

1.1 Spécification (nom de la fonction, documentation)

```
372 void tri(int n, CLIENT tab[n], int (*fonction_comparaison)(CLIENT, CLIENT)){
373     //on déclare l'indice qui sera utiliser pour les échanges
374     int j;
375     //on déclare la variable temporaire pour les échange, de même type que les éléments de la liste
376     CLIENT tmp;
377     //on parcourt le tableau
378     for (int i=1; i < n; i++) {
379         j = i;
380
381         //la fonction_comparaison renvoie -1 si tab[j-1] est après tab[j] ce qui permet de trier de manière croissante
382         while (j > 0 && fonction_comparaison(tab[j], tab[j-1]) < 0) {
383             //on fait remonter l'élément tant qu'il est inférieur
384             tmp = tab[j];
385             tab[j] = tab[j-1];
386             tab[j-1] = tmp;
387             //on met à jour j après l'échange
388             j--;
389         }
390     }
391 }
```

Entrée : Cette fonction de tri permet de trier le tableau tab de type structure CLIENT, de taille n tout deux donné en paramètre, en effectuant un tri par insertion. Ce tri prend aussi en paramètre, un pointeur de fonction de comparaison prenant en entrée 2 structures CLIENT permettant d'avoir une seule fonction de tri pour tout le programme puisque ce pointeur va permettre d'effectuer les comparaisons dans la fonction de tri.

Précondition : Le pointeur de fonction doit pointer sur une fonction de comparaison valide qui renvoie -1 quand la première structure est inférieure à la deuxième, 0 quand elles sont considérées comme égale et 1 sinon

Return : Cette fonction ne retourne rien car elle modifie directement le tableau tab

Post : Le tableau tab est trié selon une certaine comparaison donnée en paramètre grâce au pointeur

1.2 Complexité théorique (ordre de grandeur et calcul en fonction de n)

Prenons en comptes les fonctions de comparaison utilisées dans tri grâce à une pointeur de fonction (le pire cas en termes d'opérations est écrit en bas de la fonction). Les fonctions qui comparent des noms, prénom, code postal et profession sont exactement pareil, il n'y a que le type de la donnée après "client1" ou "client2" qui change (le code de ces fonctions est dans la partie de l'évaluation pratique)

Avant la boucle for (2 opérations)

Avant la boucle for on effectue : "int j;" et "CLIENT tmp;" = 2 déclarations

Boucle for (effectuée n-1 fois) -> 3 + 4 (n-1)

Les opérations dans la boucle for sont : "int i = 1 ;" = 1 déclaration + 1 initialisation, mais uniquement une seule fois, Condition "i < n" = 1 comparaison, Incrément "i++" = 1 addition + 1 assignation, "j = i;" = 1 assignation, "i < n" = 1 comparaison en fin de boucle

Boucle while (pire cas pour chaque i)

Dans le pire cas (qui est un tableau trié décroissant), la boucle while fait i boucles pour chaque boucle de la boucle for. Pour chaque boucle de while, les opérations sont les suivantes : Condition "j > 0" = 1 comparaison. Pour l'appel de la fonction de comparaison "fonction_comparaison(tab[j], tab[j-1])" : "tab[j]" et "tab[j-1]" = 2 accès aux tableaux et 1 soustraction, la fonction de comparaison = k opérations, "< 0" = 1 comparaison. L'échange des clients chaque fois que le client actuel n'est pas à sa place: "tmp = tab[j];" = 1 assignation + 1 accès, "tab[j] = tab[j-1];" = 1 assignation + 1 accès, "tab[j-1] = tmp;" = 1 assignation + 1 accès. Enfin la décrémentation "j--;" = 1 addition + 1 assignation pour un nombre total par boucle while de : $1 + 2 + k + 1 + 2 + 2 + 2 + 1 = 13 + k$

Nombre de boucles while effectuées

Pour chaque i dans la boucle for, la boucle while s'exécute jusqu'à i fois dans le pire cas. Donc, le nombre total d'itérations est la somme de i allant à n-1 avec $i = 1$ où $i = ((n-1)*n)/2$. Ainsi, le coût total des opérations dans la boucle while est : $(13+k)*((n-1)*n)/2$

Conclusion

On additionne ensuite tous les coûts précédents donc : avant la boucle for -> 2 opérations, la boucle for -> $4(n-1)+3$ opérations et la boucle while -> $(13+k)*2(n-1)n$ opérations. Ainsi le pire cas va donc dépendre des n CLIENT contenu dans le tableau de client à trier et de k qui est le nombre d'opérations de la fonction de comparaison. Ainsi tout additionné -> $2 + (4(n-1)+3) + ((13+k) \times ((n-1)n)/2)$ ce qui donne simplifié -> $(1 + 4n) + (((13+k)(n-1)n)/2)$ mais pour la complexité, on garde -> $((13+k)(n-1)n)/2$. Enfin, la complexité est quadratique, c'est-à-dire en $O(n^2)$ pour ce tri par insertion. Ainsi donc le tri sur la liste de 5000 clients, en triant sur le nom (dans le pire cas), la fonction devrait effectuer : $((13+19) \times (5000-1) \times 5000) / 2 \sim 400\,000\,000$ d'opérations, après un test sur la liste le tri effectue 242 757 655 opérations

1.3 Code pour l'évaluation pratique

Veillez zoomer le document pour mieux voir le code.

```
int tri(int a, CLIENT *tab, int (*fonction_comparaison)(CLIENT, CLIENT), int (*fonction_compteur)(CLIENT, CLIENT)) {
    //AUTEUR : Kenzo
    int compteur_operation = 0;
    int j; //! déclaration
    CLIENT tmp; //! déclaration
    compteur_operation = compteur_operation + 4; //on rajoute le "int i="
    for (int i=1; i < n; i++) {
        //! déclaration + 1 initialisation (effectue 1 seule fois) + 1 comparaison
        //+ 1 assignation + 1 addition
        j = i; //! assignation
        compteur_operation = compteur_operation + 4; // "j = i", "i++" et "i++"
        while (j > 0 && fonction_comparaison(tab[j], tab[j-1]) < 0) {
            //! comparaison + 2 accès + k le nombre d'opération de la fonction de comparaison
            //+ 1 comparaison + 1 appel de fonction
            compteur_operation = compteur_operation + 5 + fonction_compteur(tab[j], tab[j-1]);
            tmp = tab[j];
            tab[j] = tab[j-1];
            tab[j-1] = tmp;
            //! assignations + 4 accès
            j--;
            //! assignation + 1 addition
            compteur_operation = compteur_operation + 9;
        }
        //on regarde pourquoi on est sorti
        if (j > 0) {
            compteur_operation = compteur_operation + 5 + fonction_compteur(tab[j], tab[j-1]);
        }
        else {
            compteur_operation++;
        }
    }
    compteur_operation++; //fin de la boucle
    return compteur_operation;
}
```

```
int comp_nom_compteur(CLIENT client1, CLIENT client2) {
    //AUTEUR : Kenzo
    int compteur = 0;
    //on regarde si un des champ est vide
    if (strlen(client1.nom) == 0 && strlen(client2.nom) == 0) {
        //! appels + 2 accès à client1.nom + 2 accès à client2.nom + 2 comparaison
        compteur = compteur + 8;
        return compteur; // Les deux champ sont vides donc on renvoie 0
    }
    if (strlen(client1.nom) == 0) {
        compteur = compteur + 8;
    }
    else {
        compteur = compteur + 4;
    }
    compteur = compteur + 8;
    if (strlen(client2.nom) == 0) {
        //! comparaison + 1 appel + 1 comparaison
        return compteur; // le nom du client1 est vide, on dit qu'il est apres
    }
    compteur = compteur + 8;
    if (strlen(client2.nom) == 0) {
        //! comparaison + 1 appel + 1 comparaison
        return compteur; // le nom du client2 est vide, on dit que le nom du client1 est avant
    }
    //! appel + 4 accès
    return compteur + 5;
}
//pire cas 19 operations
```

```
int comp_date_compteur(CLIENT client1, CLIENT client2) {
    //AUTEUR : Kenzo
    int compteur = 0;
    //on regarde si un des champ est vide
    if (client1.date_naissance.annee == 0 && client2.date_naissance.annee == 0) {
        //! accès + 2 comparaison
        compteur = compteur + 8;
        return compteur; // Les deux champ sont vides donc on renvoie 0
    }
    if (client1.date_naissance.annee == 0) {
        compteur = compteur + 8;
    }
    else {
        compteur = compteur + 4;
    }
    compteur = compteur + 4;
    if (client1.date_naissance.annee == 0) {
        //! accès + 1 comparaison
        return compteur; // l'annee de naissance du client1 est vide, on dit qu'il est apres
    }
    compteur = compteur + 4;
    if (client2.date_naissance.annee == 0) {
        //! accès + 1 comparaison
        return compteur; // l'annee de naissance du client2 est vide, on dit que l'annee de naissance du client1 est avant
    }
    compteur = compteur + 8;
    if (client1.date_naissance.annee < client2.date_naissance.annee) {
        //! accès + 1 comparaison
        return compteur; // l'annee de naissance du client 1 est avant
    }
    compteur = compteur + 8;
    if (client1.date_naissance.annee > client2.date_naissance.annee) {
        //! accès + 1 comparaison
        return compteur; // l'annee de naissance du client 1 est apres
    }
    compteur = compteur + 8;
    // Si les annees sont pareil, on regarde les mois
    if (client1.date_naissance.mois < client2.date_naissance.mois) {
        //! accès + 1 comparaison
        return compteur; //le mois de naissance du client 1 est avant
    }
    compteur = compteur + 8;
    if (client1.date_naissance.mois > client2.date_naissance.mois) {
        //! accès + 1 comparaison
        return compteur; //le mois de naissance du client 1 est apres
    }
    // Si les mois sont identiques, comparer les jours
    compteur = compteur + 8;
    if (client1.date_naissance.jour < client2.date_naissance.jour) {
        //! accès + 1 comparaison
        return compteur; //le jour de naissance du client 1 est avant
    }
    compteur = compteur + 8;
    if (client1.date_naissance.jour > client2.date_naissance.jour) {
        //! accès + 1 comparaison
        return compteur; //le jour de naissance du client 1 est apres
    }
    // Les dates sont pareil, on renvoie 0
    return compteur;
}
//pire cas 64 operations
```

2. Algorithme de filtre

2.1 Spécification (nom de la fonction, documentation)

```
int filtre(int n, CLIENT *tab){
    //Aucun : none
    //on initialise la liste de resultat du filtre
    CLIENT *tab_filtre = malloc(n * sizeof(CLIENT)); // Allocation dynamique
    //on initialise un entier qui sera la taille du tableau apres le filtre
    //on augmente ce chiffre a chaque fois qu'on rajoute un client
    int taille_filtre = 0;
    //on declare le filtre
    char filtre[TAILLE];
    //on declare une variable qui va contenir le choix
    char choix;
    do{
        printf("Choisissez le filtre a appliquer : \n\n");
        printf("1. Filtrer sur le nom\n");
        printf("2. Filtrer sur le prenom\n");
        printf("3. Filtrer sur la profession\n");
        printf("4. Filtrer sur l'adresse\n");
        printf("5. Filtrer sur l'age\n");
        printf("\nVotre choix : ");
        scanf("%c", &choix);
        //on enleve le cache
        if(choix != '\n'){
            vider_cache();
        }
        while(choix < '1' || choix > '5');
        if (choix == '1' && choix <= '4'){
            printf("Entrez le filtre que vous voulez appliquer (199 caracteres max): ");
            fgets(filtre, TAILLE, stdin);
            //enleve le cache
            if (strcmp(filtre, "\n") == NULL){
                vider_cache();
            }
            else{
                retour_chariot(filtre);
            }
            //on enleve le saut de ligne apres la saisie
            enlever_saut_ligne(TAILLE, filtre);
            //on parcourt les clients
            for (int i = 0; i < n; i++) {
                int valide = 0;
                //on regarde en fonction du choix ce que l'on va filtrer
                if (choix == '1')
                    valide = est_compris_dans(filtre, tab[i].nom);
                else if (choix == '2')
                    valide = est_compris_dans(filtre, tab[i].prenom);
                else if (choix == '3')
                    valide = est_compris_dans(filtre, tab[i].profession);
                else if (choix == '4')
                    valide = est_compris_dans(filtre, tab[i].ville_cp);
                //Si le filtre est valide on peut l'ajouter
                if (valide) {
                    tab_filtre[taille_filtre] = tab[i];
                    taille_filtre++;
                }
            }
            //on tri le tableau selon le critere choisi
            if (choix == '1')
                tri(taille_filtre, tab_filtre, comp_nom);
            if (choix == '2')
                tri(taille_filtre, tab_filtre, comp_prenom);
            if (choix == '3')
                tri(taille_filtre, tab_filtre, comp_profession);
            if (choix == '4')
                tri(taille_filtre, tab_filtre, comp_cp);
        }
        if (choix == '5'){
            int limite_age;
            printf("Entrez l'age : ");
            scanf("%d", &limite_age);
            //on enleve le residu
            getchar();
            //on parcourt les clients
            for (int i = 0; i < n; i++) {
                //on traite uniquement les clients qui ont une date de naissance renseignee
                if (tab[i].date_naissance.jour != 0 && tab[i].date_naissance.mois != 0 && tab[i].date_naissance.annee != 0) {
                    int age = 2025 - tab[i].date_naissance.annee;
                    //on regarde si l'age correspond
                    if (age == limite_age) {
                        tab_filtre[taille_filtre] = tab[i];
                        taille_filtre++;
                    }
                }
            }
            //on tri selon la date
            tri(taille_filtre, tab_filtre, comp_date);
        }
        afficher_tous_clients(taille_filtre, tab_filtre);
        free(tab_filtre); //on libere la memoire
    }
}
```

Entrée : Filtre prend en paramètre un tableau tab de structure CLIENT, ainsi que n, un entier qui est la taille du tableau tab

Précondition : Aucune

Return : Ne renvoie rien

Post : La fonction filtre et trie le tableau tab selon un critère choisi par l'utilisateur (nom, prenom, ...) et affiche ensuite le résultat du filtre appliqué, trié sur le même champ (si on filtre sur le nom, alors on trie sur le nom)

2.2 Complexité théorique (ordre de grandeur et calcul en fonction de n)

Ici on ne prendra pas en compte le nombre d'opérations effectuer par tri et par la fonction "afficher_clients" qui permet d'afficher le résultat après le filtre effectué car ces deux fonctions sont utilisées après le filtre mais sont dans la fonction mais ici on s'intéresse à la complexité du filtrage et pas de l'affichage ou même du tri, de même pour l'affichage lorsque que l'on demande l'option de filtre ou le filtre, ainsi qu'enlever les sauts de lignes et les utilisations de "getchar".

Avant le début de la boucle principale

Avant la boucle, on effectue: "CLIENT *tab_filtre = malloc(n * sizeof(CLIENT));" = 1 appel à malloc (comme on ne connaît pas le nombre d'opération qu'effectue malloc on dira ici que c'est 1 opération), "int taille_filtre = 0;" = 1 déclaration + 1 assignation, "char filtre[TAILLE];" = 1 déclaration, "char choix;" = 1 déclaration

Do-while (choix du filtre)

Ici on néglige donc les "printf" et "vider_cache" pour cette boucle et on part du principe que l'utilisateur ne remplit qu'une seule fois le caractère. Par conséquent ici, on effectue: "scanf("%c", &choix);" = 1 lecture + 1 assignation + 1 accès, "choix < '1' || choix > '5'" = 2 comparaisons

Choix de filtrer sur le nom, le prénom, la profession ou l'adresse (les choix entre 1 et 4)

On regarde ici le nombre d'opérations de ce choix et on effectue donc: "fgets(filtre, TAILLE, stdin)" = 1 lecture + 1 assignation. On parcourt de tous les clients avec la boucle for ainsi on effectue: "int valide = 0;" = 1 déclaration (une seule fois) + 1 assignation, "choix == '1'/'2'/'3'/'4'" = 1 comparaison (donc 4 comparaisons dans le pire cas), "valide = est_compris_dans(filtre, tab[i].nom)" = 1 appel + 2 accès "tab[i]" et "tab[i].nom", "tab[i].prenom", etc... + k le nombre d'opération de la fonction + 1 assignation, "tab_filtre[taille_filtre] = tab[i];" = 2 accès "tab[i]" et "tab_filtre[taille_filtre]" + 1 assignation, "taille_filtre++;" = 1 addition + 1 assignation, "i < n" = 1 comparaison, "i++;" = 1 addition + 1 assignation. Ici on néglige aussi le message apporté à l'utilisateur si le filtre échoue puisque que on s'intéresse au filtrage et non au confort

Choix de filtrer sur l'âge

On regarde ici le nombre d'opération de ce choix là et on effectue: "scanf("%d", &limite_age)" = 1 lecture + 1 assignation + 1 accès. On parcourt de tous les clients avec la boucle for et on fait ces opérations élémentaires: "if (tab[i].date_naissance.jour != 0 && tab[i].date_naissance.mois != 0 && tab[i].date_naissance.annee != 0)" = 3 comparaisons + 9 Accès "tab[i].date_naissance" sur le jour, le mois et l'année, "Int age = 2025 - tab[i].date_naissance.annee" = 1 soustraction + 3 accès à tab[i].date_naissance + 1 assignation, "age == limite_age" = 1 comparaison, "tab_filtre[taille_filtre] = tab[i];" = 2 accès "tab_filtre[taille_filtre]" et "tab[i];" + 1 assignation, "taille_filtre++;" = 1 addition et 1 assignation, "i < n" = 1 comparaison, "i++;" = 1 addition + 1 assignation

Conclusion

Pour avoir le pire cas (en négligeant le tri, les fonctions pour enlever les caractères résiduels lors de la saisie et les différents affichage), il nous suffit donc d'additionner les opérations élémentaires suivantes: avant la boucle principale : 5 opérations, Do-while : 5 opérations, Filtrer sur le nom etc... : 2 opérations (fgets) + 1 déclaration + (n) x (1 assignation + 4 comparaisons (pire des cas) + 1 appel + k + 2 accès + 1 assignation + 2 accès + 1 assignation + 1 addition + 1 assignation + 1 addition + 1 assignation) + 1 comparaison quand on sort de la boucle. Comme on ne peut pas filtrer aussi sur l'âge, le total sur ce type de filtre => $14 + 16n + k*n$ (k le nombre d'opération de "est_compris_dans" dont le code est donné dans la partie d'après). La fonction est_compris_dans effectue -> $3 + 2k + 5 + (\text{taille du filtre}) * (14) \rightarrow 8 + 2k + 14n$ opérations (n = taille du filtre entrée et k, le nombre d'opération de strlen). Enfin le total additionné est => $14 + 16n + (8 + 2j + 14k) * n$ (n la taille du tableau donné, k la taille du filtre entrée et j, le nombre d'opération de strlen(filtre)) Enfin lorsque l'on filtre sur l'âge : 3 opérations (scanf) + (n) x (3 + 9 + 1 + 3 + 1 + 2 + 1 + 1 + 1 + 1 + 2) pour un total sur ce type de filtre de => $13 + 25n$

2.3 Code pour l'évaluation pratique

Veuillez zoomer le document pour mieux voir le code.

```
int filtre(int n, CLIENT *tab){
//AUTEUR : Menes
int compteur_operation = 0;
//on initialise la liste de requête du filtre
CLIENT *tab_filtre = malloc(n * sizeof(CLIENT)); // Allocation dynamique
//1 déclaration + 1 assignation + 1 appel + 1 multiplication + k le nombre d'opération de la fonction malloc
//on initialise un entier qui sera la taille du tableau après le filtre
//on suppose ce chiffre a chaque fois qu'on rajoute un client
int taille_filtre = 0;
//1 déclaration + 1 assignation
//on déclare le filtre
char filtre[TAILLE];
//1 déclaration
//on déclare une variable qui va contenir le choix
char choix;
//1 déclaration
compteur_operation = compteur_operation + 8;
do{
printf("Choisissez le filtre à appliquer : %s\n", "Nom");
printf("1. Filtrer sur le nom\n");
printf("2. Filtrer sur le prenom\n");
printf("3. Filtrer sur la profession\n");
printf("4. Filtrer sur l'adresse\n");
printf("5. Filtrer sur l'âge\n");
printf("votre choix : ");
scanf("%c", &choix);
//1 lecture + 1 assignation + 1 accès
compteur_operation = compteur_operation + 3;
//on enlève le cache
if(choix != '\n'){
vider_cache();
} while(choix < '1' || choix > '5');
//on passe sur le fait que cette boucle est infinie si l'utilisateur
//ne fait que d'entrer un choix mauvais et on partira du principe
//qu'elle ne s'exécute qu'une seule fois
//2 comparaisons
compteur_operation = compteur_operation + 2;

//2 comparaisons
if (choix == '1' || choix == '4'){
//on regarde le choix pour savoir le nombre d'opération à ajouter

printf("Entrez le filtre que vous voulez appliquer (199 caractères max): ");
fgets(filtre, TAILLE, stdin);
//1 lecture + 1 assignation
compteur_operation = compteur_operation + 2;
//enlève le cache
if (strcmp(filtre, "\n") == NULL){
vider_cache();
} else{
retour_chariot(filtre);
//on enlève le saut de ligne après la saisie
enlever_saut_ligne(TAILLE, filtre);

//1 déclaration + 1 assignation (une seule fois les 2)
compteur_operation = compteur_operation + 3; //on ajoute aussi la déclaration de valide
//on compare avec (n-1) assignation + 1 addition
//on parcourt les clients
for (int i = 0; i < n; i++) {
//1 déclaration (une fois) + 1 assignation
int valide = 0;
compteur_operation = compteur_operation + 4;
//on regarde en fonction du choix ce que l'on va filtrer
if (choix == '1'){
//1 comparaison
compteur_operation++;
valide = est_compris_dans(filtre, tab[i].nom);
//1 assignation + 1 appel + 2 accès + k le nombre d'opération de la fonction appelé
compteur_operation = compteur_operation + est_compris_dans_compteur(filtre, tab[i].nom);
}
else if (choix == '2'){
//1 comparaison
compteur_operation = compteur_operation + 2;
valide = est_compris_dans(filtre, tab[i].prenom);
//1 assignation + 1 appel + 2 accès + k le nombre d'opération de la fonction appelé
compteur_operation = compteur_operation + est_compris_dans_compteur(filtre, tab[i].prenom);
}
else if (choix == '3'){
//1 comparaison
compteur_operation = compteur_operation + 3;
valide = est_compris_dans(filtre, tab[i].profession);
//1 assignation + 1 appel + 2 accès + k le nombre d'opération de la fonction appelé
compteur_operation = compteur_operation + est_compris_dans_compteur(filtre, tab[i].profession);
}
else if (choix == '4'){
//1 comparaison
compteur_operation = compteur_operation + 4;
valide = est_compris_dans(filtre, tab[i].ville_cp);
//1 assignation + 1 appel + 2 accès + k le nombre d'opération de la fonction appelé
compteur_operation = compteur_operation + est_compris_dans_compteur(filtre, tab[i].ville_cp);
}
//si le filtre est valide on peut l'ajouter
if (valide) {
//2 accès + 1 assignation
compteur_operation = compteur_operation + 3;
tab_filtre[taille_filtre] = tab[i];
//1 assignation + 1 addition
taille_filtre++;
compteur_operation = compteur_operation + 2;
}
}
//on ne prend pas en compte le message affiché
//on écrit un message au cas où le filtre n'a rien trouvé
if (taille_filtre == 0) {
printf("Aucun client trouvé\n");
free(tab_filtre); //on libère la mémoire
}
//on ne prend pas en compte le tri effectué ensuite qui
//est le uniquement pour le confort utilisateur
//on tri le tableau selon le critère choisi
if (choix == '1')
tri(taille_filtre, tab_filtre, comp_nom);
if (choix == '2')
tri(taille_filtre, tab_filtre, comp_prenom);
if (choix == '3')
tri(taille_filtre, tab_filtre, comp_profession);
if (choix == '4')
tri(taille_filtre, tab_filtre, comp_cp);
}
//on regarde combien d'opération on a fait pour le if au dessus
if (choix == '1'){
compteur_operation = compteur_operation + 2;
}
else
compteur_operation++;
}
```



```
if (choix == '5'){
    //1 comparaison
    int limite_age;
    //1 déclaration
    compteur_operation = compteur_operation + 2;
    printf("Entrez l'age : ");
    scanf("%d", &limite_age);
    //1 assignation + 1 accès + 1 lecture
    compteur_operation = compteur_operation + 3;
    //on enlève le residu
    getchar();
    //on parcourt les clients
    //1 déclaration + 1 assignation (une seule fois les 2)
    compteur_operation = compteur_operation + 3; //on ajoute aussi la déclaration de age
    //n comparaison = (n-1)*1 assignation + 1 addition)
    for (int i = 0; i < n; i++) {
        compteur_operation = compteur_operation + 3;
        //on traite uniquement les clients qui ont une date de naissance renseignée
        if (tab[i].date_naissance.jour != 0 && tab[i].date_naissance.mois != 0 && tab[i].date_naissance.annee != 0) {
            if (tab[i].date_naissance.mois != 0)
                compteur_operation = compteur_operation + 12;
            else
                compteur_operation = compteur_operation + 8;
            int age = 2025 - tab[i].date_naissance.annee;
            //1 déclaration(une seule fois) + 1 assignation + 1 soustraction + 3 accès
            compteur_operation = compteur_operation + 5;
            //on regarde si l'age correspond
            //1 comparaison
            compteur_operation +=
                if (age == limite_age){
                    //2 accès + 3 assignation
                    tab_filtre[taille_filtre] = tab[i];
                    //2 addition + 1 assignation
                    taille_filtre++;
                    compteur_operation = compteur_operation + 5;
                }
            else{
                compteur_operation = compteur_operation + 4; //la première condition à été regardé
            }
        }
        compteur_operation++; //on sort de la boucle
        //on tri selon la date
        tri(taille_filtre, tab_filtre, comp_date);
    }
    afficher_tous_clients(taille_filtre, tab_filtre);
    free(tab_filtre); //on libère la memoire
    //la n° nombre d'opération de la fonction + 1 appel
    compteur_operation++;
    return compteur_operation;
}

int est_compris_dans_compteur(char filtre[], char chaine[]){
    //AUTEUR : Kenzo
    //on calcul la taille de filtre
    int compteur = 0;
    //1 déclaration + 1 appel + k le nombre d'opération de strlen + 1 assignation
    int tailleF = strlen(filtre);
    compteur = compteur + 3;
    //on regarde si le mot n'est pas trop petit par rapport au filtre
    //1 appel + 1 comparaison
    compteur = compteur + 2;
    if (strlen(chaine) < tailleF){
        //on ne prend pas le mot
        return compteur;
    }
    else{
        //on parcourt le filtre et on verifie que toutes les lettres de filtre
        //sont dans mot et dans l'ordre
        //(1 déclaration + 1 assignation) une seule fois + 1 comparaison tailleF fois +
        //(1 assignation + 1 addition) tailleF-1 fois
        compteur = compteur + 2;
        for (int i=0; i<tailleF; i++){
            compteur = compteur + 3;
            //6 accès + 3 comparaison + 1 addition + 1 soustraction
            if (filtre[i] != chaine[i] && filtre[i] + 32 != chaine[i] && filtre[i] - 32 != chaine[i]){
                compteur = compteur + 7; //première condition forcément vrai et deuxième forcément regardé
                if (filtre[i] + 32 != chaine[i])
                    compteur = compteur + 4;
                //si les caracteres sont different on ne prend pas le mot
                return compteur;
            }
            else{
                compteur = compteur + 3; //première condition regardé
            }
        }
        compteur++; // on sort de la boucle
        //si tout a été verifié on peut garder le mot
        return compteur;
    }
}
```

3. Algorithmes de recherches

3.1 Spécification formelle

L'algorithme de recherche que nous avons préconisé est séparé en deux recherches différentes : l'une où l'on demande la saisie de l'adresse ou du numéro de téléphone de la personne recherchée (fonction : recherche_client), et l'autre où l'on insiste sur le numéro de téléphone, clé primaire de la base (fonction : recherche_client_tel). Elles prennent en paramètres toutes les deux le nombre de lignes du fichier, et le tableau qui contient tous les clients. Les deux fonctions de recherche traitent tous les indices du tableau passé en paramètre jusqu'à atteindre la personne recherchée ou la fin du fichier.

3.2 Complexité théorique des fonctions de recherche

Dans le cas général, nous devons traiter le nombre de boucles présent et en déduire l'ordre de grandeur de la complexité. La fonction recherche_client ne présente que des saisies utilisateurs et une seule boucle. Le reste de celle-ci ne comporte que des conditions (if... else). De plus, la boucle présente est une boucle while qui s'arrête à n personnes si la recherche aboutie, ou le nombre de lignes du fichier si la recherche n'a pas abouti.

Voici une partie de la `while(i<nb_lignes && !trouve){`
boucle while :

La première
vérification stipule que la
recherche continue tant que le programme n'a pas atteint la fin du tableau. La
deuxième vérification fait en sorte de continuer le programme tant que le client n'a pas
été trouvé.

```
//on verifie si le prenom de la persc
if(strcmp(tab[i].prenom, p) == 0){
//on verifie si le nom de la pers
if(strcmp(tab[i].nom, n) == 0){
```

La fonction `recherche_client_tel` ne voit pas sa complexité théorique changée, elle contient globalement le même `while`.

La complexité théorique est alors linéaire, de grandeur $O(\text{nb_lignes})$. Les fonctions dépendent alors de la longueur du tableau. Il est intéressant à noter que la recherche est donc une recherche simple, et linéaire, ce qui peut s'avérer assez pratique et rapide si l'on veut traiter un long fichier (comme l'annuaire de 5000 personnes) sans pour autant consommer énormément de mémoire.

3.3 Dénombrement des opérations élémentaires des fonctions de recherche

Bien que les deux fonctions de recherche aient la même complexité théorique, elles n'admettent pas le même nombre d'opérations élémentaires. LES OPERATIONS ELEMENTAIRES SERONT DENOMBREES DANS LE PIRE DES CAS (recherche aboutie et personne recherchée en fin de tableau).

3.3.1 Dénombrement sur la fonction `recherche_client`

Commençons par la fonction `recherche_client`. Nous trouvons 3 déclarations de variables qui sont affectées (3 opérations). Ensuite, 5 chaînes de caractères sont déclarées avec un accès de `TAILLE` (8 opérations). Dans les lignes suivantes, nous avons un `while` qui prend deux conditions : chaque tour, on a 3 opérations élémentaires qui s'ajoutent N fois (N correspond au nombre de lignes du fichier), soit $N*3$. On a, pour l'instant $N*3+8$ opérations élémentaires. Ensuite, dans les prochaines lignes, il y a 3 `if` : un en fonction du prénom (p), un en fonction du nom (n), un qui regarde sur l'utilisateur a tapé 'adresse' pour chercher l'adresse mail ou pas. A noter qu'il y a aussi un accès de tableau avec `tab[i].x` et une comparaison avec 0. `Stricmp` compare caractère par caractère autrement dit : $p+2+n+2+\text{num_ou_ad}+2$ soit $N*(3+p+2)+n+\text{num_ou_ad}+12$ (la comparaison avec le prénom est toujours répétée).

Ici, les choix se démultiplie : si l'utilisateur cherche par l'adresse, on a une recherche entre l'adresse renseignée (`ad_mel`) et celui d'une personne de `tab` avec 2 opérations comme avant en plus. Nous avons : $N*(3+p+2)+\text{num_ou_ad}+\text{ad_mel}+14$.

Si l'utilisateur est trouvé, indice et trouve s'initialisent, ce qui rajoute 2 opérations élémentaires soit : $N*(3+p+2)+n+num_ou_ad+ad_mel+16$.

Si l'utilisateur recherche par le numéro de telephone, on a presque le même nombre d'opérations or il s'agira de `num_tel` au lieu de `ad_mel` soit

$$N*(3+p+4)+n+num_ou_ad+num_tel+16.$$

Dans chaque tour du while, `i` s'incrmente, soit 2 opérations élémentaires en plus. A la fin du while, les conditions échouent donc on a le nombre d'opérations élémentaires en plus, soit 3 (l'incrmentation de `i` se multiplie avec `N`). Si le client est trouvé, il y a une opération élémentaire, sinon il y a en a un total de 2 à cause d'un `printf` en plus (pour nous, le client sera trouvé, donc on prend le 1). On a un total de 4 opérations élémentaires.

Pour la recherche par adresse mail, on a $N*(3+p+4)+num_ou_ad+ad_mel+20$, et pour téléphone, nous avons $N*(3+p+4)+n+num_ou_ad+num_tel+20$

Avant de commencer la recherche, il faut s'assurer que le prénom n'est pas présent en double : ajoutons un prénom non existant dans la dernière ligne de l'annuaire 50 entrées. Vérifions avec le code (code présent dans la dernière partie du fichier). Prenons un client trouvable sur la toute dernière personne du fichier annuaire 50 entrées, pour que chaque if se fasse, avec prénom, nom, adresse_mail : Thomas, Fontaine-Boulay, thomas.fontaine-boulay@gmail.com (`num_ou_ad` vaudra 7 car il aura 'adresse'). Le nombre d'opérations élémentaires devrait être de : $50*(3+6+4)+15+7+32+20 = 724$.

ATTENTION : BIEN METTRE UNE MULTIPLICATION ENTRE LE N ET LE P CAR LA COMPARAISON EST TOUJOURS FAITE !

Pour les autres cas, il faut adapter le calcul, voici ce que la fonction renvoie :

```
724
Process returned 0 (0x0)   execution time : 0.184 s
Press any key to continue.
```

La formule est donc correcte.

3.3.2 Dénombrement sur la fonction recherche_client_tel

La fonction recherche_client_tel a presque la même complexité. Ce qui change, c'est qu'on ne demande pas de taper "adresse" au client, et on ne vérifie pas l'adresse mail.

On ne déclare pas l'adresse mail, ni num_ou_ad, ce qui diminue de 2 le nombre d'opérations élémentaires. Sans la vérification de num_ou_ad, il y a 2 opérations élémentaires en moins, avec un total de 4 opérations élémentaires à enlever.

On a alors (en dénombrant) : $N \cdot (3 + p + 4) + n + 2 + \text{num_tel} + 2 + 12$ soit $N \cdot (3 + p + 4) + n + \text{num_tel} + 16$

Le calcul est alors très similaire et s'écrit : $N \cdot (3 + p + 4) + n + \text{num_tel} + 16$. Reprenons le fichier annuaire 50 avec les mêmes données qu'avant sauf qu'on place le numéro de téléphone au lieu de l'adresse mail soit : Thomas, Fontaine-Boulay, 04.66.87.48.30. On devrait avoir $50 \cdot (3 + 6 + 4) + 15 + 14 + 16 = 695$.

Voici ce que renvoie l'appel à la fonction :

```
695
Process returned 0 (0x0)   execution time : 0.146 s
Press any key to continue.
```

Le calcul est donc correct.

3.3 Code des fonctions d'opérations formelles

Veuillez zoomer le document afin de mieux voir les codes.

Code de recherche_client :

```
int recherche_client_ope(int nb_lignes, CLIENT *tab){
    //AUTEUR : Ilia
    int i=0, trouve = 0, indice = -1, op=3; //3 affectations
    char p[]="Thomas", n[]="Fontaine-Boulay", ad_mel[]="thomas.fontaine-boulay@gmail.com", num_ou_ad[]="adresse", num_tel[TAILLE2]
    op+=5; //5 declarations avec acces
    while(i<nb_lignes && !trouve){
        op+=3; //2 comparaisons et 1 'et'
        op+=strlen(p)+2; //une recherche, 1 acces, une comparaison
        if(strcmp(tab[i].prenom, p) == 0){
            op+=strlen(n)+2; //une recherche, 1 acces, une comparaison
            if(strcmp(tab[i].nom, n) == 0){
                op+=strlen(num_ou_ad)+2; //une recherche, une comparaison
                if(strcmp(num_ou_ad, "adresse") == 0){
                    op+=strlen(ad_mel)+2; //une recherche, 1 acces, une comparaison
                    if(strcmp(tab[i].adresse_mel, ad_mel) == 0){
                        trouve = 1;
                        op+=1; //1 operation
                        indice = i;
                        op+=1; //1 operation
                    }
                }
            }
        }
        else{
            op+=strlen(num_tel)+2; //une recherche, 1 acces, une comparaison
            if(strcmp(tab[i].telephone, num_tel) == 0){
                trouve = 1;
                op+=1; //1 operation
                indice = i;
                op+=1; //1 operation
            }
        }
    }
}
```

```
    }  
    }  
    }  
    i++;  
    op+=2; //une operation-affectation  
    }  
    op+=3; //while echoue  
    op+=1; //1 comparaison  
    if(indice != -1);  
    else{  
        printf("Client introuvable\n");  
        op+=1; //1 printf  
    }  
    return op;  
}
```

Code de recherche_client_tel :

```
int recherche_client_tel_ope(int nb_lignes, CLIENT *tab){  
    //AUTEUR : Ilian  
    int i=0, trouve = 0, indice = -1, op=3; //3 affectations  
    char p[]="Thomas", n[]="Fontaine-Boulay", num_tel[]="04.66.87.48.30";  
    op+=3; //3 declarations avec acces  
    while(i<nb_lignes && !trouve){  
        printf("%d, %d\n", op, i);  
        op+=3; //2 comparaisons et 1 'et'  
        op+=strlen(p)+2; //une recherche, 1 acces, une comparaison  
        if(stricmp(tab[i].prenom, p) == 0){  
            op+=strlen(n)+2; //une recherche, 1 acces, une comparaison  
            if(stricmp(tab[i].nom, n) == 0){  
                op+=strlen(num_tel)+2; //une recherche, 1 acces, une comparaison  
                if(strcmp(tab[i].telephone, num_tel) == 0){  
                    trouve = 1;  
                    op+=1; //1 operation  
                    indice = i;  
                    op+=1; //1 operation  
                }  
            }  
        }  
        i++;  
        op+=2; //une operation-affectation  
    }  
    op+=3; //while echoue  
    op+=1; //1 comparaison  
    if(indice != -1);  
    else{  
        printf("Client introuvable\n");  
        op+=1; //1 printf  
    }  
    return op;  
}
```
